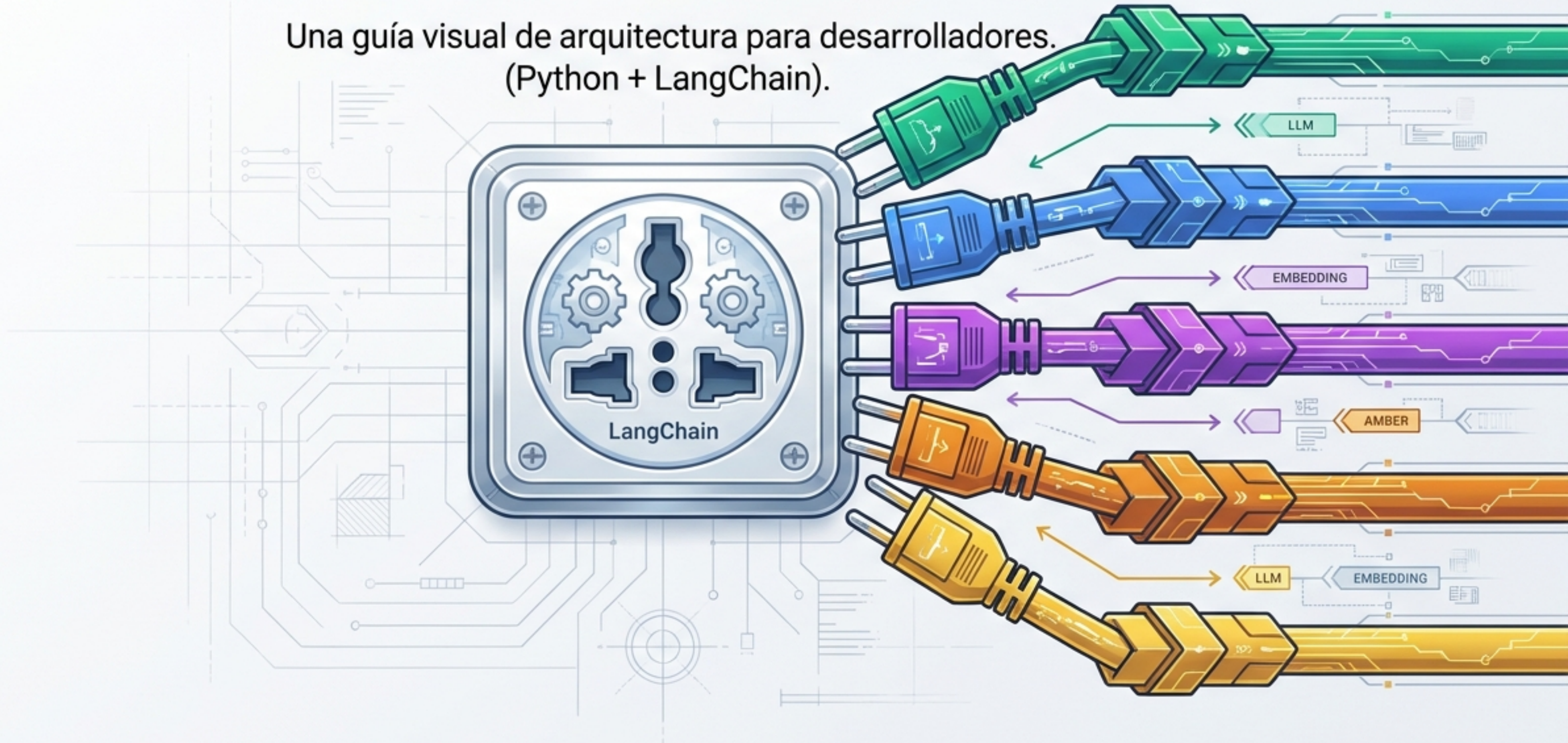


EL ADAPTADOR UNIVERSAL

5 Formas de Conectar LLMs y Embeddings con LangChain.

Una guía visual de arquitectura para desarrolladores.
(Python + LangChain).





Chroma



FAISS

Lo que vamos a resolver (y lo que no).

Esta no es una clase sobre bases de datos vectoriales. Es un manual definitivo para dominar la capa de conexión entre tu código y los modelos.

El objetivo: Conectar el cliente con el servidor.



RAG



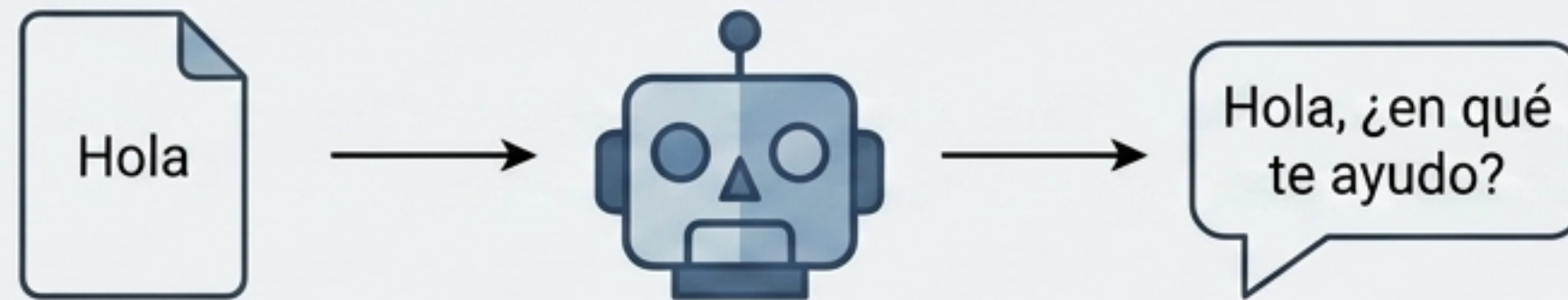
Chunking



Cosine
Similarity

La idea central: Son dos modelos, dos conexiones distintas. Conceptualmente, no son intercambiables. Necesitas instanciar dos objetos diferentes en LangChain.

1. LLM / Chat Model



"Hola" → [Generación] → "Hola, ¿en qué te ayudo?"

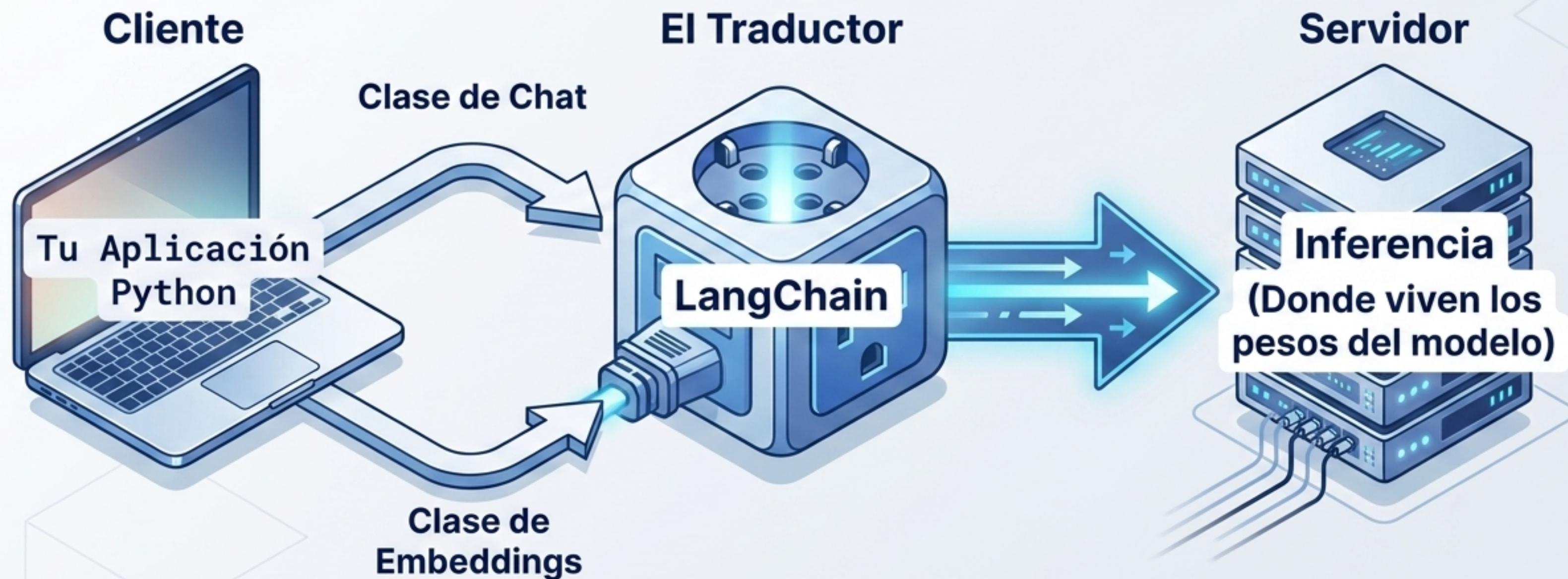
2. Embedding Model



"Hola" → [Vectorización] → [0.1, -0.4, 0.8]

La abstracción: Separando el cliente del servidor.

LangChain coloca una capa traductora. Tú hablas con LangChain; LangChain habla con el mundo.



Las tres variables que controlan todo el sistema.



model

El Motor. ¿Qué modelo específico quiero utilizar? (ej. gpt-4o). No indica dónde vive el servidor.



api_key

La Credencial. ¿Cómo me identifico?
Permite que el servidor autorice la request.



base_url

Las Coordenadas. ¿A qué servidor físico envió la request? (Clave para servidores locales o gateways).

Tres patrones arquitectónicos para conectar la inferencia.

Patrón Nativo Remoto

Clases específicas que se conectan directamente a las APIs del creador del modelo. (OpenAI, Gemini).



Gateway Compatible

Aprovecha contratos compatibles (estilo OpenAI) para unificar facturación y acceso. (OpenRouter).



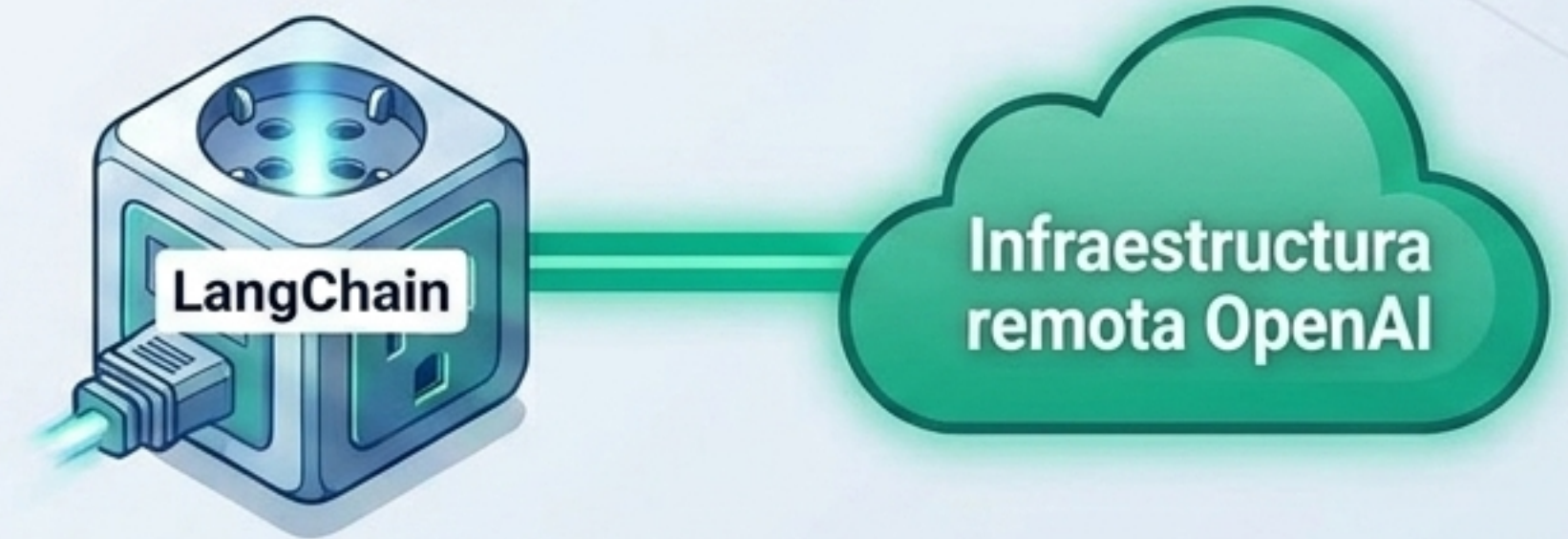
Ejecución Local

El servidor de inferencia vive en tu propio hardware. Sin costos de nube, mayor privacidad. (LM Studio, Hugging Face).



Forma 1: OpenAI (El Patrón Nativo Directo).

```
llm = ChatOpenAI(model="gpt-4o")  
embeddings = OpenAIEmbeddings(  
    model="text-embedding-3-small")
```

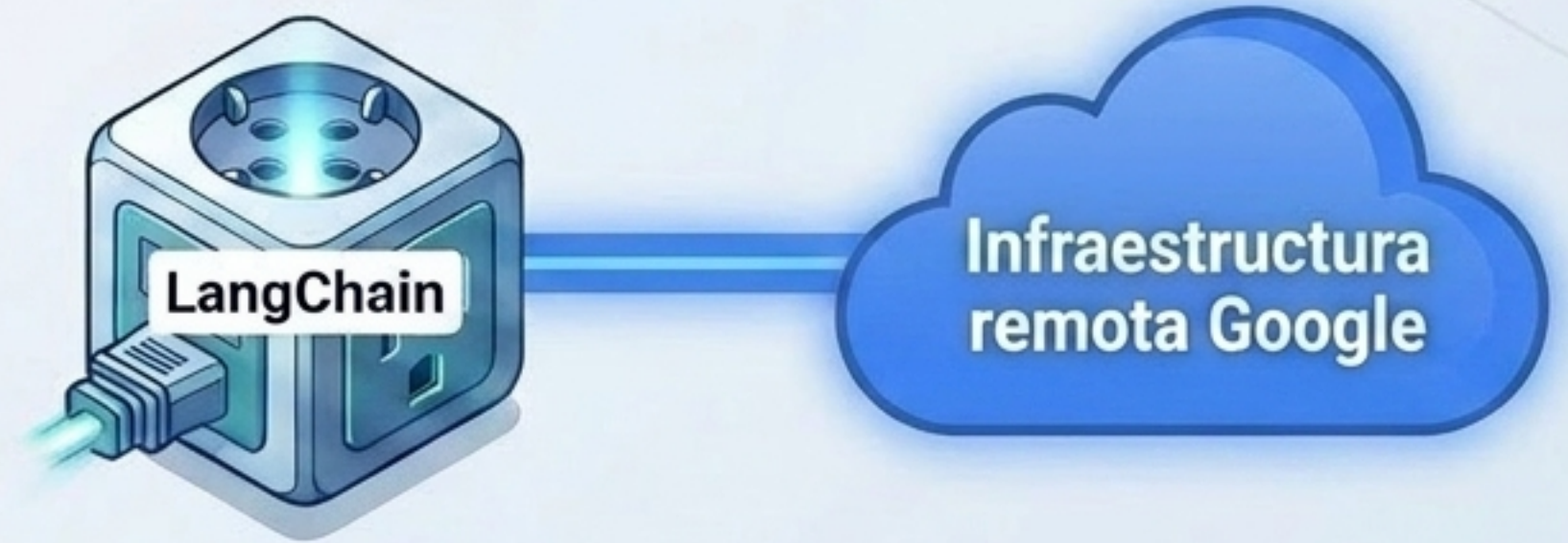


¿Dónde ocurre la inferencia?

En la nube. Tu computadora no carga los pesos del modelo.

Forma 2: Google Gemini (Ecosistemas Nativos Distintos).

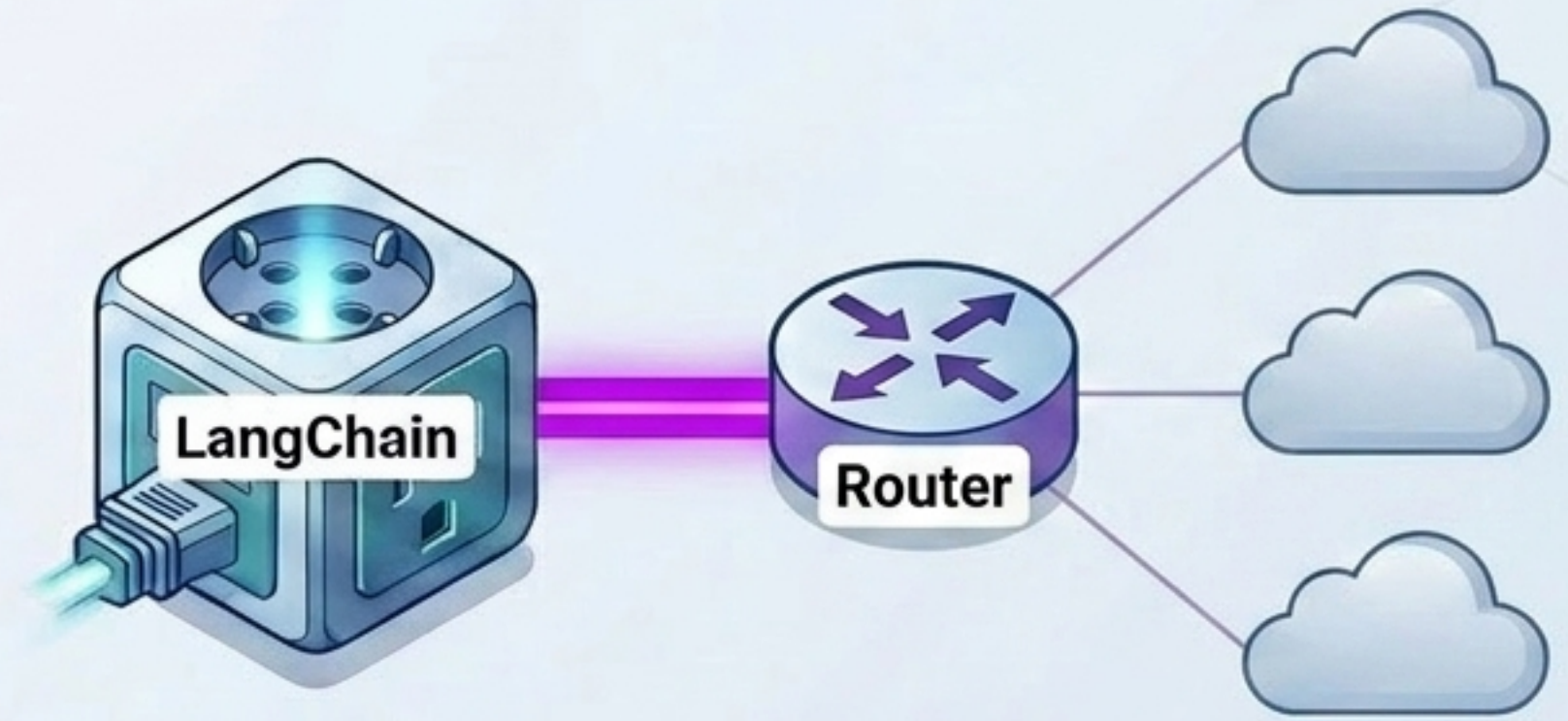
```
llm = ChatGoogleGenerativeAI(  
    model="gemini-1.5-flash")  
  
embeddings =  
    GoogleGenerativeAIEmbeddings(  
        model="models/embedding-001")
```



Aunque ambos sean Gemini, no usamos el mismo objeto. El modelo no en tu máquina, ocurre vía Google API.

Forma 3: OpenRouter (El Gateway Multiplexor).

```
llm = ChatOpenRouter(  
    model="anthropic/claude-3-haiku")  
  
embeddings = OpenAIEmbeddings(  
    base_url="https://openrouter.ai/  
api/v1")
```



¡Mira los embeddings!

Usamos la clase `OpenAIEmbeddings` pero el servidor es OpenRouter. ¿Por qué? Porque OpenRouter ofrece un contrato compatible. El `base_url` decide a dónde llega realmente la request.

Forma 4: LM Studio (El Flip Conceptual Local).

```
llm = ChatOpenAI(base_url="http://localhost:1234/v1", api_key="lm-studio")
```



Forma 5: Hugging Face (El Ecosistema Híbrido).



A diferencia de LM Studio (que requiere un servidor separado corriendo), el Pipeline local de HF carga los pesos directamente en tu proceso de Python. No hay request HTTP.

La Matriz Arquitectónica.

Forma	Clase LLM	Clase Embeddings	¿API key?	¿base_url manual?
OpenAI	ChatOpenAI	OpenAIEmbeddings	Sí	No
Gemini	ChatGoogleGenerativeAI	GoogleGenerativeAIEmbeddings	Sí	No
OpenRouter	ChatOpenRouter	OpenAIEmbeddings	Sí	Sí (para embeddings)
LM Studio	ChatOpenAI	OpenAIEmbeddings	Valor ficticio	Sí (a localhost)
Hugging Face	ChatHuggingFace	HuggingFaceEmbeddings	Depende	No (para local)

El Mapa de Infraestructura: ¿Dónde ocurre la inferencia?

Nube Pública



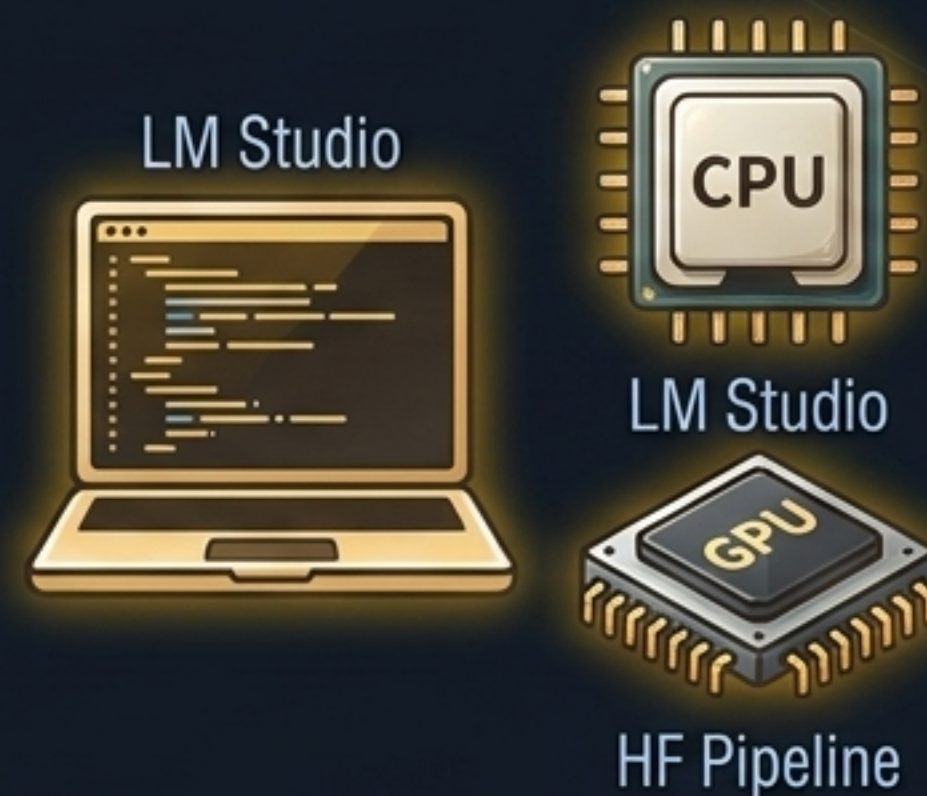
Inferencia remota directa.
Costo por token.

Detrás del Router



Gateway en la nube.
Te abstraes del
proveedor final.

Tu Hardware



El costo computacional pasa
a tu máquina (CPU/GPU
local). Privacidad total.

Diagnóstico de Errores Frecuentes.

 **ERROR (Incorrecto)**

 **ARQUITECTURA CORRECTA**

 Usar un modelo de chat como embedding porque 'es el mismo LLM'.

 Requieres un modelo de texto y un modelo de vectorización distintos.

 Creer que la clase ChatOpenAI siempre envía datos a OpenAI.

 Si alteras el base_url, ChatOpenAI puede hablar con tu propia laptop (LM Studio).

 Pasar temperature a la clase de Embeddings.

 temperature es para generación (LLM). Los embeddings son deterministas.

Lo que cambia y lo que no cambia (El Patrón Factory).



Cambia:

- La función que inicializa la conexión
- `def get_llm(): ...`

NO Cambia:

- La lógica de tu aplicación.
- `llm = get_llm()`
- `llm.invoke('Hola')`

LangChain estandariza la interfaz. Puedes cambiar el cartucho del proveedor (Local, Nube, Gateway) sin romper el resto de tu código.